

Technical Report :

Blockchain-Based Library Book Lending Tracker

Technical Report: Individual Assignment 1

Student: Joseph Nishimwe

Date: 03 October 2026

GitHub: https://github.com/josep-prog/formative_introduction_blockchain

Demo video: <https://youtu.be/cwhcAC3NmaE>

File Name	Link	Purpose of the file
DemoVideo	DemoVideo	Full explanation of the project and a guide on how to run it.
pending.c	pending.c	Holds the pending pool: the list of signed blocks waiting to be mined. It adds a block to the pool, removes it once mined, and grows the list in memory as needed.
pending.h	pending.h	Defines the pending pool structure and

		declares its functions.
ledger.c	ledger.c	Tracks the tokens in both transaction models. UTXO: unspent outputs, change outputs and double-spend checks. Account-based: balances, nonces and a per-account transaction history stored as a linked list. It also applies the 1-coin fee and rejects transfers with insufficient balance.
ledger.h	ledger.h	Defines the transaction, UTXO and account structures and declares the ledger functions.
Mining.c	mining.c	Confirms pending blocks onto the chain in three ways: solo, pool (reward shared by work done, 2% pool fee) and cloud rental (rental and maintenance fees, profit check). It also runs the

		proof-of-work difficulty benchmark.
Mining.h	mining.h	Declares the mining functions and sets the mining constants, such as the 50-coin block reward.
Blockchain.c	blockchain.c	Manages the blockchain: creates the genesis, borrow, return and overdue blocks, links them with hashes, finds active loans, validates the chain, and saves and loads chain.txt.
Blockchain.h	blockchain.h	Defines the Block structure and declares the blockchain functions.
Crypto.c	crypto.c	Handles security with OpenSSL: makes the key pair, saves the encrypted private key and the public key, signs and verifies blocks, and hashes librarian PINs.
Crypto.h	crypto.h	Declares the cryptographic functions used by main.c and blockchain.c.

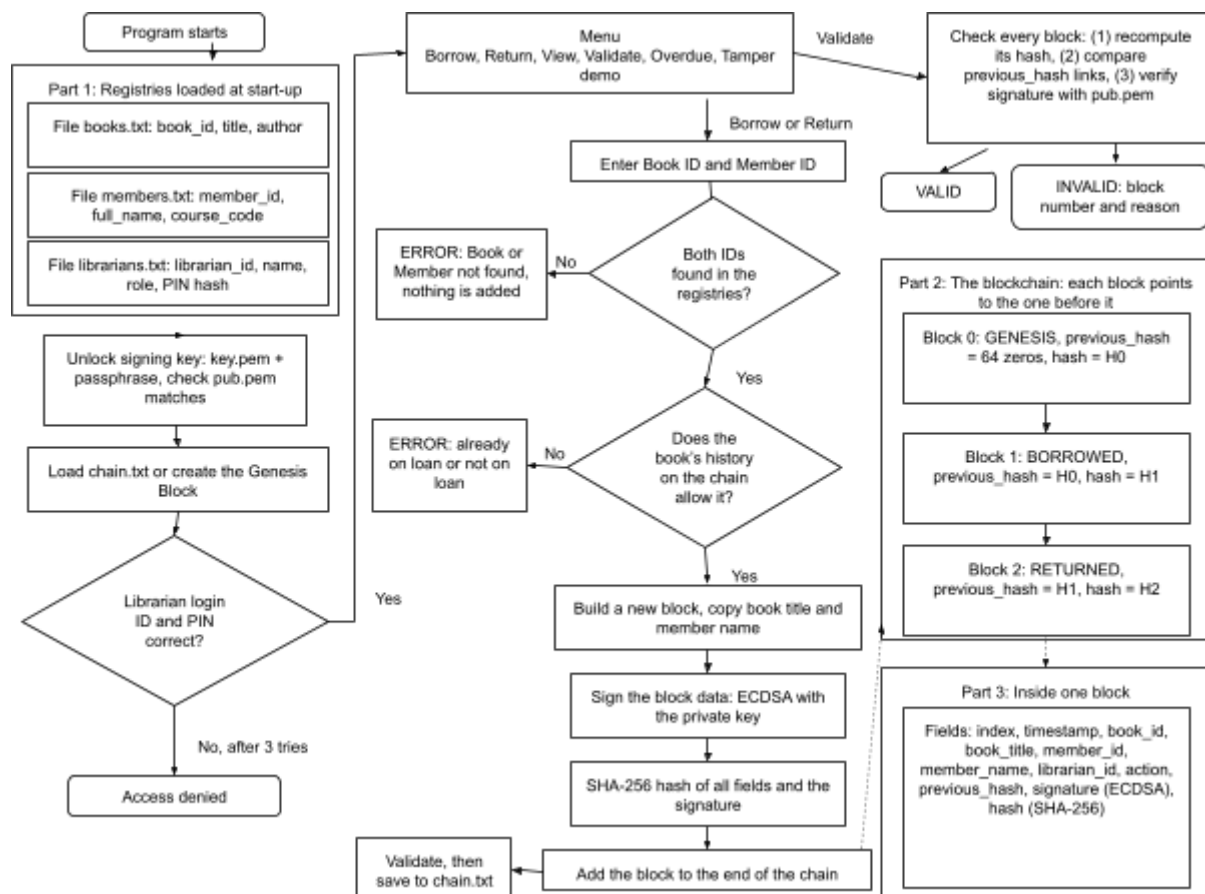
Main.c	main.c	Controls the whole program: the login, the menu, and calling the other files to load records, create blocks and run security checks.
Registry.h	registry.h	Defines the Book, Member and Librarian structures and declares the registry functions.
Registry.c	registry.c	Loads books, members and librarians from the data files, reports bad lines, and finds a record by its ID.
Books.txt	books.txt	The book registry: ID, title and author of each book.
Members.txt	members.txt	The member registry: ID, full name and course code of each member.
Librarians.txt	librarians.txt	Staff who can log in: ID, name, role (ADMIN or LIBRARIAN) and a PBKDF2 hash of their PIN.
Makefile	Makefile	Builds the whole program and links OpenSSL with one

		command (make).
--	--	-----------------

1. Introduction

This project is a small library system that records every book loan on a blockchain, so that no record can be quietly changed afterwards.

A normal library keeps its loans in a notebook or a database. Whoever controls that notebook can edit or erase a line, and nobody would know. Here, each borrow and each return becomes a sealed, signed record that is linked to the one before it. Changing an old record breaks the link, and the program notices.



The system does three things:

- **It records lending.** A librarian logs in, then records who borrowed or returned which book.
- **It secures the records.** Each record is signed, then "mined" into the chain using Proof of Work.

- **It rewards good behaviour.** A member who returns a book on time earns 10 coins, which can be sent to other members.

The program is written in C and runs in a terminal. Every screenshot in this report comes from real runs of the program, so every number can be traced back to an earlier step.

2. Blockchain in one minute

A blockchain is a list of records where each record carries the fingerprint of the one before it, so the past cannot be edited without being noticed.

Six words are enough to follow the rest of this report:

Word	What it means in plain English	In this project
Block	One sealed record	One borrow, return or overdue event
Hash	A 64-character fingerprint of some data. Change one letter and the fingerprint changes completely	SHA-256
Chain	Each block stores the hash of the block before it	<code>previous_hash</code> field
Signature	Proof of who created a record, which cannot be forged without a secret key	The librarian's ECDSA signature
Mining	Doing a deliberate amount of computer	Proof of Work, difficulty 1 to 4

Word	What it means in plain English	In this project
	work before a block is accepted	
Token	A coin that exists only as an entry in the records	Reward for returning a book

Every lending record goes through the same five steps:

1. **Record.** The librarian enters a book ID and a member ID.
2. **Sign.** The program signs the record with the library's private key.
3. **Wait.** The signed record sits in a waiting area called the pending pool.
4. **Mine.** A miner does the Proof of Work and links the record to the end of the chain.
5. **Reward.** Only now is any token reward paid to the member.

Sections 4 to 8 follow these steps in order, using real runs of the program.

3. Project structure

The code is split into seven small parts, each with one job, so that the library logic stays separate from the blockchain and the cryptography.

```

joe@code:~/Templates/formative_introduction_blockchain$ ls
data library Makefile README.md src Technical_Report_Library_Blockchain-FormativeAssignment.pdf
joe@code:~/Templates/formative_introduction_blockchain$ tree
.
├── data
│   ├── books.txt
│   ├── key.pem
│   ├── librarians.txt
│   ├── members.txt
│   └── pub.pem
├── library
├── Makefile
├── README.md
├── src
│   ├── blockchain.c
│   ├── blockchain.h
│   ├── crypto.c
│   ├── crypto.h
│   ├── ledger.c
│   ├── ledger.h
│   ├── main.c
│   ├── mining.c
│   ├── mining.h
│   ├── pending.c
│   ├── pending.h
│   ├── registry.c
│   └── registry.h
└── Technical_Report_Library_Blockchain-FormativeAssignment.pdf

3 directories, 22 files
joe@code:~/Templates/formative_introduction_blockchain$ tree

```

The project folder: `src` holds the code, `data` holds the registries and keys, and `library` is the compiled program.

File	Its one job
main.c	Shows the menu, handles the login and calls the other parts
registry.c	Loads books, members and librarians, and finds a record by its ID
blockchain.c	Creates blocks, links them, runs the Proof of Work and validates the chain
crypto.c	Makes keys, signs and verifies records, hashes PINs (using OpenSSL)
pending.c	Holds the pending pool of signed blocks waiting to be mined

File	Its one job
mining.c	Solo, pool and cloud mining, plus the difficulty benchmark
ledger.c	Tracks tokens in both models: UTXO and account-based

The data folder holds five books, five members and two librarians as plain text files. It also holds the signing key (key.pem, locked with a passphrase) and the public key (pub.pem) that anyone can use to check signatures.

Building and starting the program

The program needs gcc, make and OpenSSL 3. One command builds it and one starts it:

```
make clean && make
./library
```

```
joe@code:~/Templates/formative_introduction_blockchain$ ./library
Loaded 5 books, 5 members and 2 librarians.
Digital signing key loaded from data/key.pem.
Started a new blockchain (genesis mined at difficulty 2).
Librarian ID: LIB001
PIN:
Welcome, Joseph Nishimwe (ADMIN).

Choose the transaction model for this session:
1. UTXO model
2. Account-based model
Model: 1
Transaction model: UTXO. 0 confirmed rewards replayed from the chain.

LIBRARY MENU [model: UTXO | difficulty: 2 | pending: 0 | chain: 1 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
7. Mine pending blocks: cloud rental
-- Tokens --
8. View confirmed lending records
9. View token balances and UTXO set
10. Transfer tokens
11. Replay last transfer (double-spend / nonce test)
12. Transaction history for a member
Chain
```

Start-up. The program loads the registries and the signing key, creates the first (genesis) block, asks the librarian to log in, then asks which token model to use.

The status line above the menu is worth watching through the rest of this report. It always shows the active model, the mining difficulty, how many

records are waiting, and how long the chain is. Here it reads pending: 0 | chain: 1 blocks: nothing is waiting, and the chain holds only the genesis block.

4. Borrowing and returning a book

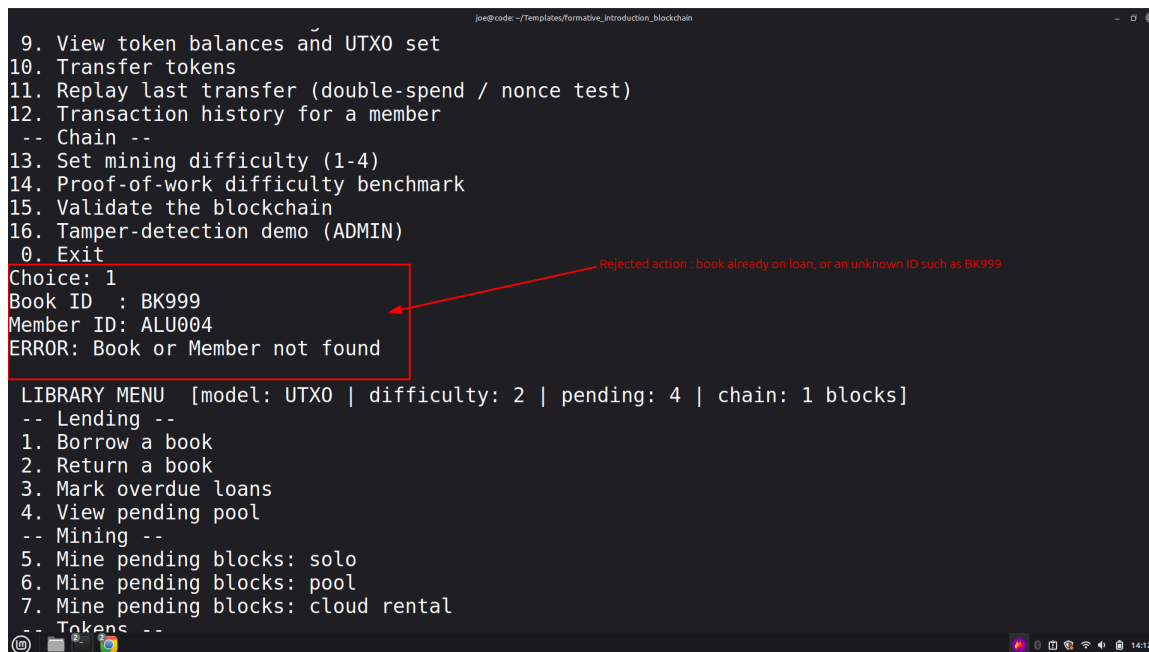
A borrow or a return is recorded and signed immediately, but it is not yet part of the blockchain: it waits to be mined.

Borrowing

The librarian chooses option 1 and types a book ID and a member ID. Before creating anything, the program checks three things:

- the book exists in the book registry,
- the member exists in the member registry,
- the book is not already out on loan.

If any check fails, the program prints an error and records nothing. If all pass, it builds a BORROWED record and signs it.



```
joe@code: ~/Templates/formative_introduction_blockchain
9. View token balances and UTXO set
10. Transfer tokens
11. Replay last transfer (double-spend / nonce test)
12. Transaction history for a member
-- Chain --
13. Set mining difficulty (1-4)
14. Proof-of-work difficulty benchmark
15. Validate the blockchain
16. Tamper-detection demo (ADMIN)
0. Exit
Choice: 1
Book ID : BK999
Member ID: ALU004
ERROR: Book or Member not found
LIBRARY MENU [model: UTXO | difficulty: 2 | pending: 4 | chain: 1 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
7. Mine pending blocks: cloud rental
-- Tokens --
```

A rejected action. Book BK999 does not exist, so the program prints an error and the pending count does not change.

```
12. Transaction history for a member
-- Chain --
13. Set mining difficulty (1-4)
14. Proof-of-work difficulty benchmark
15. Validate the blockchain
16. Tamper-detection demo (ADMIN)
0. Exit
Choice: 1
Book ID : BK001
Member ID: ALU001
BORROWED block is PENDING (1 in the pool) - mine the pool to confirm it.
Borrow of 'The Money Trap: Lost Illusions Inside the Tech Bubble' for Irakoze Jean recorded.

LIBRARY MENU [model: UTXO | difficulty: 2 | pending: 1 | chain: 1 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
7. Mine pending blocks: cloud rental
-- Tokens --
8. View confirmed lending records
9. View token balances and UTXO set
10. Transfer tokens
11. Replay last transfer (double-spend / nonce test)
12. Transaction history for a member
-- Chain --
13. Set mining difficulty (1-4)
```

Irakoze Jean (ALU001) borrows "The Money Trap" (BK001). The status line now reads pending: 1, while the chain is still 1 block long.

Returning

A return works the same way, with two extra rules. The book must actually be on loan, and only the member who borrowed it can return it.

The program then checks the time. The loan period is 120 seconds by default, kept short so a demonstration does not take two weeks.

What happened	Reward
Returned on time	10 coins
Returned late	5 coins
Never returned (marked overdue)	No reward

```
17. Proof-of-work difficulty benchmark
15. Validate the blockchain
16. Tamper-detection demo (ADMIN)
0. Exit
Choice: 2
Book ID : BK001
Member ID: ALU001
Returned ON TIME after 56 seconds: 10-coin reward.
Reward transaction created: 10 coins to ALU001 (TX 0d89bf9f0442d7c0...).
RETURNED block is PENDING (2 in the pool) - mine the pool to confirm it.
Return of 'The Money Trap: Lost Illusions Inside the Tech Bubble' recorded.

LIBRARY MENU [model: UTXO | difficulty: 2 | pending: 2 | chain: 1 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
7. Mine pending blocks: cloud rental
-- Tokens --
8. View confirmed lending records
9. View token balances and UTXO set
10. Transfer tokens
11. Replay last transfer (double-spend / nonce test)
12. Transaction history for a member
-- Chain --
13. Set mining difficulty (1-4)
14. Proof-of-work difficulty benchmark
```

The book comes back after 56 seconds, inside the loan period, so a 10-coin reward transaction is created for ALU001.

Notice the wording in Figure 5: the reward transaction is *created*, not *paid*. The coins reach the member only after the block is mined. The reward amount is also covered by the librarian's signature, so nobody can raise it afterwards.

Two more loans were then recorded in the same session, so that the pool had several records to mine.

```
joe@code: ~/Templates/formative_introduction_blockchain
0. Exit
Choice: 1
Book ID : BK003
Member ID: ALU003
BORROWED block is PENDING (4 in the pool) - mine the pool to confirm it.
Borrow of 'Sustainable Leadership' for Uwase Diane recorded.

LIBRARY MENU [model: UTXO | difficulty: 2 | pending: 4 | chain: 1 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
7. Mine pending blocks: cloud rental
-- Tokens --
8. View confirmed lending records
9. View token balances and UTXO set
10. Transfer tokens
11. Replay last transfer (double-spend / nonce test)
12. Transaction history for a member
-- Chain --
13. Set mining difficulty (1-4)
14. Proof-of-work difficulty benchmark
15. Validate the blockchain
16. Tamper-detection demo (ADMIN)
0. Exit
Choice:
```

Uwase Diane (ALU003) borrows "Sustainable Leadership" (BK003). Four records are now waiting.

Overdue loans

Option 3 checks every book that is still out. Any loan older than the loan period gets an OVERDUE record, which carries no reward.

```
joe@code: ~/Templates/formative_introduction_blockchain
10. Transfer tokens
11. Replay last transfer (double-spend / nonce test)
12. Transaction history for a member
-- Chain --
13. Set mining difficulty (1-4)
14. Proof-of-work difficulty benchmark
15. Validate the blockchain
16. Tamper-detection demo (ADMIN)
0. Exit
Choice: 3
OVERDUE block is PENDING (5 in the pool) - mine the pool to confirm it.
OVERDUE: 'Wars Guns & Votes: Democracy in Dangerous Places' borrowed by Joseph Habimana (ALU002) - no token transaction.
OVERDUE block is PENDING (6 in the pool) - mine the pool to confirm it.
OVERDUE: 'Sustainable Leadership' borrowed by Uwase Diane (ALU003) - no token transaction.
2 loan(s) marked overdue (loan period: 120 seconds).

LIBRARY MENU [model: UTXO | difficulty: 2 | pending: 6 | chain: 1 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
```

Two loans pass the 120-second period and are marked overdue. Each one adds a record to the pool, taking it from 4 to 6, and neither creates a token transaction.

5. The pending pool

The pending pool is the waiting room: every signed record sits here until a miner confirms it.

Option 4 lists everything that is waiting. Think of it as a tray of signed forms on the librarian's desk. Each form is genuine, but none has been filed in the permanent archive yet.

```
joe@code: ~/Templates/formative_introduction_blockchain
13. Set mining difficulty (1-4)
14. Proof-of-work difficulty benchmark
15. Validate the blockchain
16. Tamper-detection demo (ADMIN)
0. Exit
Choice: 4
Pending pool: 4 unconfirmed blocks waiting to be mined
# Action Book Member Reward By Reward TX Signature
-----
1 BORROWED BK001 Irakoze Jean 0 LIB001 (none) VALID
2 RETURNED BK001 Irakoze Jean 10 LIB001 0d89bf9f0442d7c0... VALID
3 BORROWED BK002 Joseph Habimana 0 LIB001 (none) VALID
4 BORROWED BK003 Uwase Diane 0 LIB001 (none) VALID
LIBRARY MENU [model: UTX0 | difficulty: 2 | pending: 4 | chain: 1 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
7. Mine pending blocks: cloud rental
-- Tokens --
8. View confirmed lending records
9. View token balances and UTX0 set
10. Transfer tokens
11. Replay last transfer (double-spend / nonce test)
```

The pending pool after four actions. All four signatures are VALID.

Reading the table from left to right:

- **Action, Book, Member** say what happened and to whom.
- **Reward** is 10 for the on-time return and 0 for the borrows. Borrowing earns nothing.
- **By** is the librarian who recorded it (LIB001).
- **Reward TX** is the fingerprint of the reward transaction. Only the return has one.

- **Signature** confirms the record has not been altered since the librarian signed it.

The pool also protects against a subtle mistake. When checking whether a book is free, the program looks at both the chain and the pool. A book cannot be lent twice just because its first loan is still waiting to be mined.

shows the other side: the confirmed chain. Before any mining, it holds only the genesis block.

```

14. Proof-of-work difficulty benchmark
15. Validate the blockchain
16. Tamper-detection demo (ADMIN)
0. Exit
Choice: 8
Confirmed chain (1 blocks):
-----
Block #0 [GENESIS]
Time      : Sat Oct  3 14:10:31 2026
PoW       : difficulty 2, nonce 280
Hash      : 00c07b9edef7e350dfc94bfad79e7132e36a16cbda8a4d5952ad2aca4464b9a9
Prev      : 0000000000000000000000000000000000000000000000000000000000000000
Signature: n/a (genesis block)

LIBRARY MENU [model: UTX0 | difficulty: 2 | pending: 6 | chain: 1 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
7. Mine pending blocks: cloud rental
-- Tokens --

```

Six records are waiting (pending: 6), yet the confirmed chain still has one block. Nothing is permanent until it is mined.

The genesis block is the fixed starting point of every chain. It has no block before it, so its Prev field is 64 zeros, and it needs no signature because it holds no lending data.

6. Mining and Proof of Work

Mining turns a waiting record into a permanent one by making the computer solve a puzzle that is slow to solve but instant to check.

The puzzle

The program adds a number, called the **nonce**, to the block and calculates the block's hash. If the hash does not start with enough zeros, it adds 1 to the nonce and tries again. The **difficulty** is the number of zeros required.

It works like a combination lock. Finding the combination takes many tries. Checking that a given combination opens the lock takes one.

Solo mining

With option 5, the logged-in librarian mines every waiting block alone.

```
Choice: 5
=== SOLO MINING by LIB001 (difficulty 2: hash must start with 2 zeros) ===
Mined block #1 after 67 hash attempts (nonce 66, 1.0 ms)
-----
Block #1 [BORROWED]
Book   : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
Member : Irakoze Jean (ALU001)
By      : LIB001
Reward  : 0 coins
TX ID   : (no transaction)
Time    : Sat Oct 3 13:40:11 2026
PoW     : difficulty 2, nonce 66
Hash    : 008c87113af95fb41176087f6c76b8a8877c941d03dcf234541e60f53d924e03
Prev    : 0097323ab5b699155d9c937539da573147aad3d5d16deb79856191ca2167d08d
Signature: VALID (72 bytes)
No reward transaction for this BORROWED block.

UTX0 set (0 unspent outputs):
TX ID           Out  Owner           Amount
-----
Total supply                0.00

Mined block #2 after 40 hash attempts (nonce 39, 1.7 ms)
-----
Block #2 [RETURNED]
Book   : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
Member : Irakoze Jean (ALU001)
```

Solo mining at difficulty 2. Block #1 needed 67 attempts (1.0 ms) and block #2 needed 40.

Three details in Figure 10 show the chain doing its job:

- **Hash** starts with 00, which is exactly what difficulty 2 demands.
- **Prev** is the hash of the genesis block. This is the link that makes it a chain.
- **Signature: VALID** means the record is still the one the librarian signed.

The borrow block carries no reward, so the token list under it is still empty. After all four blocks were mined, the status line read pending: 0 | chain: 5 blocks.

The miner earns 50 coins for each block confirmed. The program also offers two other ways to mine the same pool:

Option	How it works	Who gets the 50 coins per block
5. Solo	One miner does all the work	The miner, in full
6. Pool	2 to 8 simulated miners work in parallel	Shared by work done, after a 2% pool fee
7. Cloud rental	Rent mining power for 1 to 5 rounds	The renter, minus a rental fee and a 10% maintenance fee

Pool mining

In a pool, several miners share the work and share the reward. Each simulated miner gets a random speed, called its hash rate. In a second run of the program, four miners mined a pool of six records.

```
je@code: ~/Templates/formative_introduction_blockchain
0. Exit
Choice: 6
Number of pool miners (2-8) [4]: 4

Pool mining reward table
=== POOL MINING with 4 miners (difficulty 2) ===
P00L-M1 hash rate: 162 attempts/round
P00L-M2 hash rate: 259 attempts/round
P00L-M3 hash rate: 390 attempts/round
P00L-M4 hash rate: 88 attempts/round

Block #1 found by P00L-M3 in round 1 (nonce 477)

-----
Block #1 [BORROWED]
Book : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
Member : Irakoze Jean (ALU001)
By : LIB001
Reward : 0 coins
TX ID : (no transaction)
Time : Sat Oct 3 14:10:57 2026
PoW : difficulty 2, nonce 477
Hash : 00ed3ce81ca691c8eb9fa6d1a5e1649e0f80b0e7a8ca652adc0d95e2cbf91687
Prev : 00c07b9edef7e350dfc94bfad79e7132e36a16cbda8a4d5952ad2aca4464b9a9
Signature: VALID (71 bytes)
No reward transaction for this BORROWED block.
```

Four miners with different speeds. POOL-M3 is the fastest at 390 attempts per round and finds block #1.

```
Block #2 found by P00L-M3 in round 1 (nonce 629)

-----
Block #2 [RETURNED]
Book : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
Member : Irakoze Jean (ALU001)
By : LIB001
Reward : 10 coins
TX ID : dc925d819d3db630e1956061aacde1bbbc1488a944f07c92b16f9f8fe1999380
Time : Sat Oct 3 14:11:21 2026
PoW : difficulty 2, nonce 629
Hash : 00cd8025eb458f5580192bff01d4c1ec6cf6740e0a65a3f498b88174ec732ac7
Prev : 00ed3ce81ca691c8eb9fa6d1a5e1649e0f80b0e7a8ca652adc0d95e2cbf91687
Signature: VALID (71 bytes)
Reward TX confirmed: 10 coins - 1.00 fee -> 9.00 credited to ALU001
```

Block #2 is the return. The last line is the moment the reward is finally paid: 10 coins, minus the 1-coin fee, so 9 are credited to ALU001.

```
UTX0 set (2 unspent outputs):
TX ID          Out  Owner          Amount
-----
dc925d819d3db630...  0  ALU001          9.00
dc925d819d3db630...  1  LIBRARY         1.00
Total supply                10.00

--- Pool reward sharing ---
Blocks confirmed : 6
Gross reward      : 300.00 coins
Pool fee (2%)     : 6.00 coins -> POOL-OP
To distribute     : 294.00 coins

+-----+-----+-----+-----+-----+-----+
| Miner ID | Hash rate | Attempts | Blocks | Share % | Reward |
+-----+-----+-----+-----+-----+-----+
| POOL-M1  | 162      | 200      | 1      | 17.97%  | 52.83  |
| POOL-M2  | 259      | 321      | 0      | 28.84%  | 84.79  |
| POOL-M3  | 390      | 486      | 5      | 43.67%  | 128.38 |
| POOL-M4  | 88       | 106      | 0      | 9.52%   | 28.00  |
+-----+-----+-----+-----+-----+-----+
| TOTAL    |          | 1113     | 6      | 100.00% | 294.00 |
+-----+-----+-----+-----+-----+-----+

LIBRARY MENU [model: UTX0 | difficulty: 2 | pending: 0 | chain: 7 blocks]
```

Six blocks earn 300 coins. The pool keeps 2% (6 coins) and shares the remaining 294.

The table in Figure 13 shows why miners join pools. Payment follows work done, not luck. POOL-M2 found no blocks at all, yet earns 84.79 coins because it did 28.84% of the attempts. POOL-M3 found five of the six blocks but takes only its 43.67% share.

Cloud rental

Cloud mining means paying a fixed fee to rent someone else's mining power. It only pays off if the rented machine confirms enough blocks to cover the fee.

```
Choice: 7
Renter account [LIB001]: ALU001
Rental duration in rounds (1-5): 4
Rental fee per round in coins [25]: 7

=== CLOUD MINING: ALU001 rents 600 hashes/round for 4 rounds at 7.00 coins/round ===
Difficulty 2, 0 pending blocks, maintenance 10% of each round's reward.

--- Cloud mining earnings ---
+-----+-----+-----+-----+-----+-----+-----+-----+
| Round | Blocks | Gross | Rental | Maintenance | Net | Cumul. net |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 0 | 0.00 | 7.00 | 0.00 | -7.00 | -7.00 | <- fees exceed rewards
| 2 | 0 | 0.00 | 7.00 | 0.00 | -7.00 | -14.00 | <- fees exceed rewards
| 3 | 0 | 0.00 | 7.00 | 0.00 | -7.00 | -21.00 | <- fees exceed rewards
| 4 | 0 | 0.00 | 7.00 | 0.00 | -7.00 | -28.00 | <- fees exceed rewards
+-----+-----+-----+-----+-----+-----+-----+
Gross earnings : 0.00 coins
Total fees paid : 28.00 coins (rental 28.00 + maintenance 0.00)
Net profit : -28.00 coins
Credited to ALU001 : 0.00 coins (gross minus maintenance)
Rental deducted from ALU001 : 9.00 coins -> CLOUD-OP
```

ALU001 rents mining power for 4 rounds at 7 coins a round. The pool is empty, so no blocks are mined and the rental loses 28 coins.

This run shows the risk plainly. With nothing to mine, the gross earnings are zero while the fees keep adding up, and the program flags every round where fees exceed rewards. ALU001 held only 9 coins, so only 9 could be deducted. A balance can never go below zero.

How much harder is each extra zero?

Option 14 mines the same test block at every difficulty and reports the effort.

```

16. Tamper-detection demo (ADMIN)
0. Exit
Choice: 14

Proof-of-work effort by difficulty (same test block each time):
Difficulty Target      Attempts    Time (ms)  Hash
-----
1      0...              19         0.3  058fc6cac3ce4403...
2      00...             162         2.0  009c38d11dcb049f...
3      000...            1890        23.7  000b8943748d55ad...
4      0000...           16763       62.3  00004d8e02f4532b...
Each extra zero multiplies the expected work by 16 (one hex digit).

LIBRARY MENU [model: UTXO | difficulty: 2 | pending: 0 | chain: 5 blocks]
-- Lending --
1. Borrow a book
2. Return a book
3. Mark overdue loans
4. View pending pool
-- Mining --
5. Mine pending blocks: solo
6. Mine pending blocks: pool
7. Mine pending blocks: cloud rental
-- Tokens --
8. View confirmed lending records
9. View token balances and UTXO set
10. Transfer tokens
11. Replay last transfer (double-spend / nonce test)
12. Transaction history for a member
-- Chain --
13. Confirming difficulty / 1 A\

```

Attempts rise from 19 at difficulty 1 to 16,763 at difficulty 4.

Each extra zero multiplies the expected work by 16, because a hash character has 16 possible values. In this run the steps were roughly 9 times, 12 times and 9 times, which is normal: mining is partly luck, so single runs land above or below the average.

This cost is the point of Proof of Work. To change an old record, a cheat would have to redo the puzzle for that block and for every block after it.

7. Tokens: two ways to keep the books

The program can track coins in two different ways, UTXO or account-based, and the librarian picks one at start-up (Figure 2).

	UTXO model	Account-based model
Everyday picture	Cash: separate notes in a wallet	A bank account: one running balance
What is stored	Every unspent "note" and its owner	One balance per member

	UTXO model	Account-based model
Your balance is	The sum of the notes you own	A single number
Paying	Hand over whole notes, get change back	Subtract from one balance, add to another
Stops spending twice by	Removing a note once it is spent	A counter (nonce) that must go up by one each time
Used by	Bitcoin	Ethereum

UTXO stands for Unspent Transaction Output. The session in this report used the UTXO model.

Balances after mining

Option 9 shows each account's balance and, underneath, the individual "notes" that make it up.

```

12. Transaction history for a member
-- Chain --
13. Set mining difficulty (1-4)
14. Proof-of-work difficulty benchmark
15. Validate the blockchain
16. Tamper-detection demo (ADMIN)
0. Exit
Choice: 9

Account      Balance
-----
ALU001       9.00
ALU002       0.00
ALU003       0.00
ALU004       0.00
ALU005       0.00
LIBRARY      1.00
LIB001      200.00

UTXO set (3 unspent outputs):
TX ID      Out  Owner      Amount
-----
0d89bf9f0442d7c0...  0  ALU001      9.00
0d89bf9f0442d7c0...  1  LIBRARY      1.00
51c99b4c4cca9ebb...  0  LIB001     200.00
Total supply      210.00

LIBRARY MENU [model: UTXO | difficulty: 2 | pending: 0 | chain: 5 blocks]
-- Lending --
1. Borrow a book
2. Return a book

```

Balances after the four blocks were mined. Total supply is 210 coins.

Every number in Figure 16 can be traced back to an earlier step:

- **ALU001 has 9.** The on-time return earned 10 coins, and a fixed 1-coin fee was taken.
- **LIBRARY has 1.** That is the fee, paid into the library's treasury.
- **LIB001 has 200.** The librarian mined four blocks at 50 coins each.

Sending tokens

Option 10 moves coins between members. Here ALU001 sends 3 coins to ALU002.

[image: Terminal showing a transfer of 3 coins from ALU001 to ALU002 with a change output of 5 and a fee of 1]

One 9-coin note goes in, three notes come out.

This is the cash picture in action. ALU001 owns a single 9-coin note and cannot tear it. The whole note is spent and replaced by three new

The old 9-coin note has disappeared from the list in. That is how the UTXO model prevents a **double spend**: a note that no longer exists cannot be spent again. Total supply is still 210, because a transfer moves coins and never creates them.

Option 11 tests this on purpose by replaying the last transfer. The UTXO model rejects it as a double spend, and the account-based model rejects it for reusing a nonce. If the sender cannot cover the amount plus the fee, the transfer is refused as well.

The same steps in the account-based model

Restarting the program with the account-based model gives the bank-account view of the same chain.

[image: Terminal showing account balances with a nonce column in the account-based model]

One balance and one nonce per account. There is no list of notes underneath.

ALU001's 9 coins and the library's 1-coin fee were rebuilt from the chain at start-up. Mining rewards are not stored on the chain, which is why LIB001 shows 0 here.

[image: Terminal showing a transfer of 1 coin from ALU001 to ALU002 with nonce 0 accepted]

ALU001 sends 1 coin to ALU002 using nonce 0. With the 1-coin fee, the balance goes from 9 to 7.

There is no change output this time. The program simply subtracts 2 from one balance and adds 1 to another. The nonce is the transaction's serial number: ALU001's first payment must carry nonce 0, the next must carry 1, and so on.

[image: Terminal showing the transaction history for ALU001 with the reward and the transfer]

Option 12 lists every transaction for one account: the 9-coin reward (sender COINBASE, meaning newly created coins) and the 1-coin transfer.

[image: Terminal showing a replayed transfer rejected because the nonce was already used]

Option 11 resubmits the same transfer. It is rejected: nonce 0 was already used and the account now expects nonce 1.

Figure 21 is the account model's defence against paying twice. Without the nonce, anyone holding a copy of an old payment could submit it again and drain the account.

8. Validating the blockchain

Validation answers one question, "has anything been changed?", by re-checking every block from the first to the last.

[image: Terminal showing option 15 reporting that the blockchain is valid]

All five blocks pass every check.

For each block, option 15 confirms that:

1. **The position is right.** Block numbers run in order, starting from a genuine genesis block.
2. **The hash is honest.** A freshly calculated fingerprint matches the stored one.
3. **The work was done.** The hash starts with the number of zeros its difficulty required.
4. **The link holds.** The block's `previous_hash` equals the hash of the block before it.
5. **The signature is genuine.** It verifies against the library's public key.
6. **The reward is untouched.** The reward and its transaction ID match the block.

If any check fails, the program names the block and the reason. Option 16, for administrators only, demonstrates this by altering a block in memory and running the same checks.

[image: Terminal showing the tamper detection demo reporting the blockchain as invalid at block 1]

The book title in block #1 is changed without re-hashing or re-signing. Validation fails at once and names the block and the reason.

Two safeguards follow from validation. The program refuses to save a chain that fails it, so a tampered version never reaches the disk. And if a saved chain fails at start-up, no rewards are replayed, so a forged reward is never paid.

Checking needs only the public key. Anyone can run `./library --verify` with no login and no passphrase, which is what makes the records verifiable by outsiders and not only by the library.

[image: Shell showing the verify command checking seven blocks with the public key and reporting the chain valid]

`./library --verify` checks all 7 blocks of the saved chain using only `pub.pem`.

Across two short runs the system recorded lending events, mined them into a chain by solo and pool mining, paid a reward, moved tokens under both models, and caught a deliberate tampering attempt.

Each part has a clear role. The registries decide what is valid. Signatures prove who recorded each event. The pending pool holds records until they are confirmed. Proof of Work makes history expensive to rewrite. Validation checks all of it at any time, using only the public key.

Limitations

- **One signing key for the whole library.** Each block names its librarian, but separate keys per librarian would be stronger.
- **Members hold no keys.** The librarian submits transfers on a member's behalf.
- **Some balances last only for the session.** Member rewards are rebuilt from the chain at start-up, but mining rewards and manual transfers are not stored on it.
- **The pending pool lives in memory.** Records still waiting when the program closes are lost, and the program warns before exiting.
- **Mining is simulated on one computer.** Pool and cloud mining imitate several miners; there is no real network.